
RI204

Windows programiranje

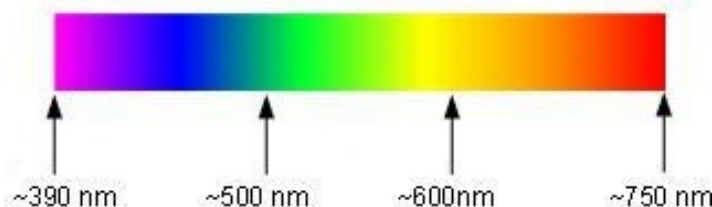
dr.sci. Edin Pjanić

Pregled predavanja

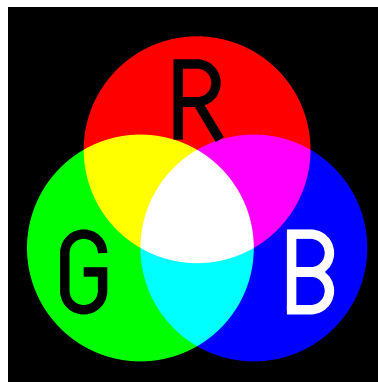
- Grafika nastavak
 - Boje
 - Grafički primitivi
 - Tačke i linije
 - Pravougaonici
 - Poligoni
 - Elipse
 - GDI objekti
 - Olovka (Pen)
 - Četkica (Brush)

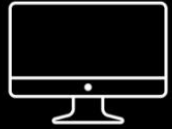
Boje - fizikalni aspekt

- Svjetlost je prezentacija elektromagnetnih talasa sa talasnom dužinom između 400 nm i 750 nm
- Boja je posljedica načina na koji ljudsko oko i mozak procesira ove talase:

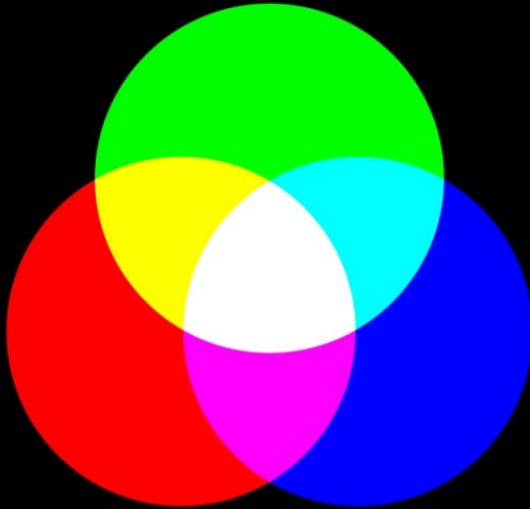


- Ranih 1900-tih otkriveno je da se svaka boja može reprodukovati kombinovanjem izvora svjetlosti sa talasnim dužinama crvene (red), zelene (green) i plave (blue), različitih intenziteta. RGB je aditivni model boja (additive color model), za razliku od CMYK koji je subtraktivni (subtractive).





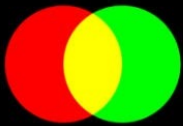
FOR SCREENS



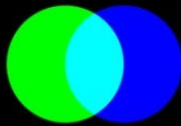
RGB

ADDITIVE COLOR SYNTHESIS

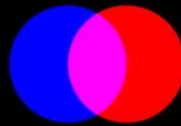
RED • GREEN • BLUE



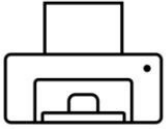
RED + GREEN



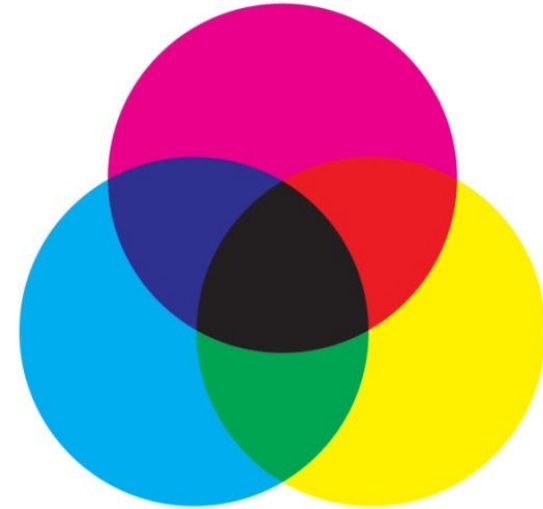
GREEN + BLUE



BLUE + RED



FOR PRINT



CMYK

SUBTRACTIVE COLOR SYNTHESIS

CYAN • MAGENTA • YELLOW • BLACK



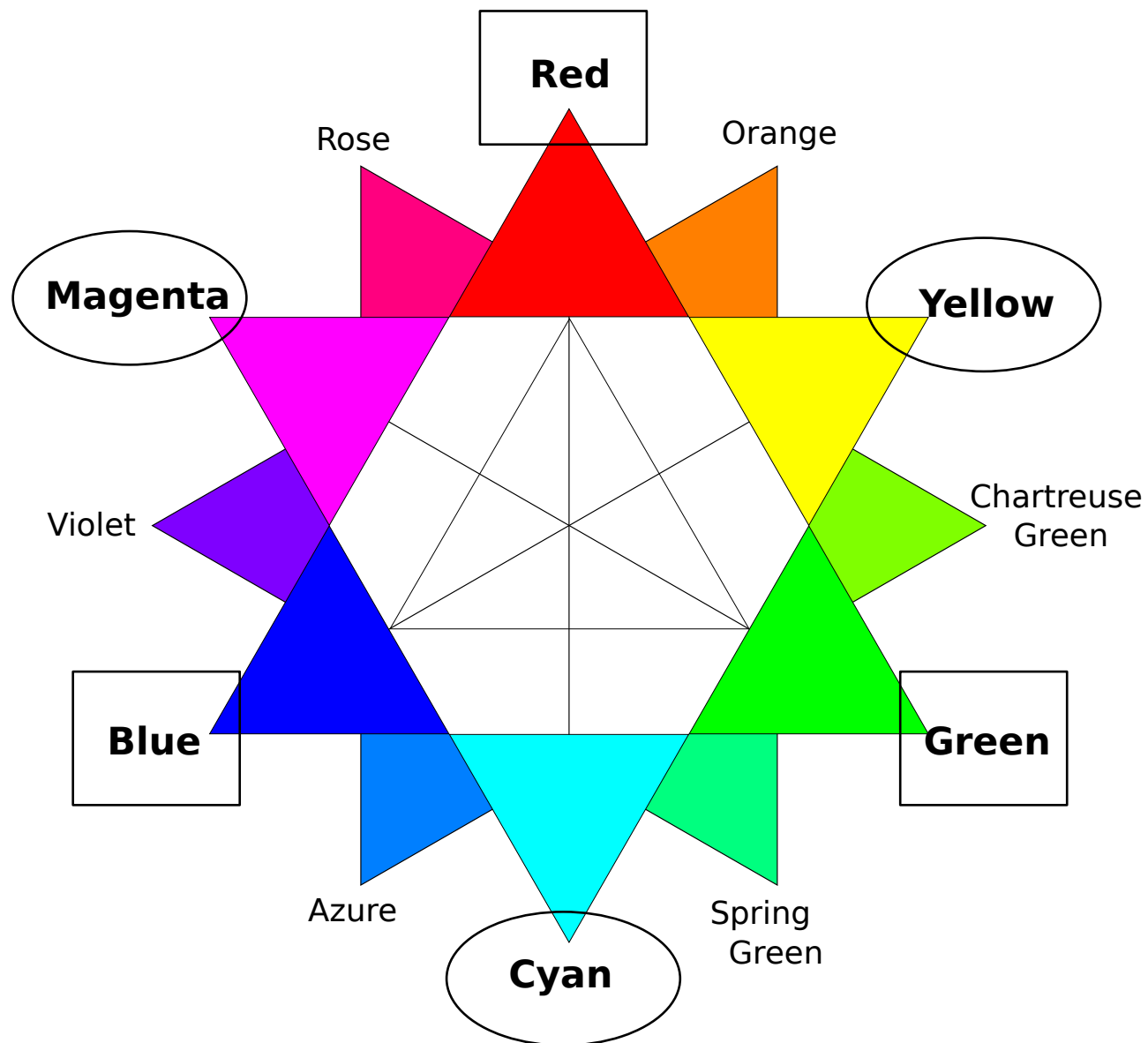
CYAN + MAGENTA



MAGENTA + YELLOW

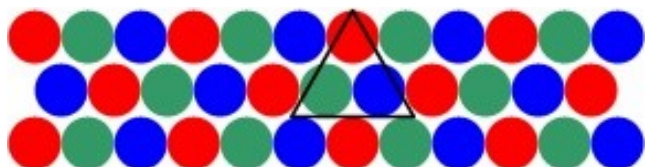


YELLOW + CYAN

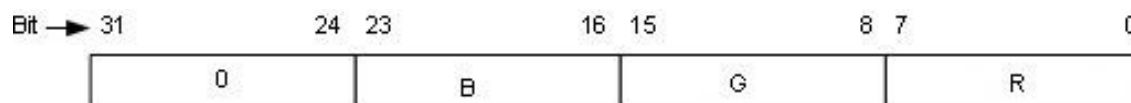


Reprodukcija boja

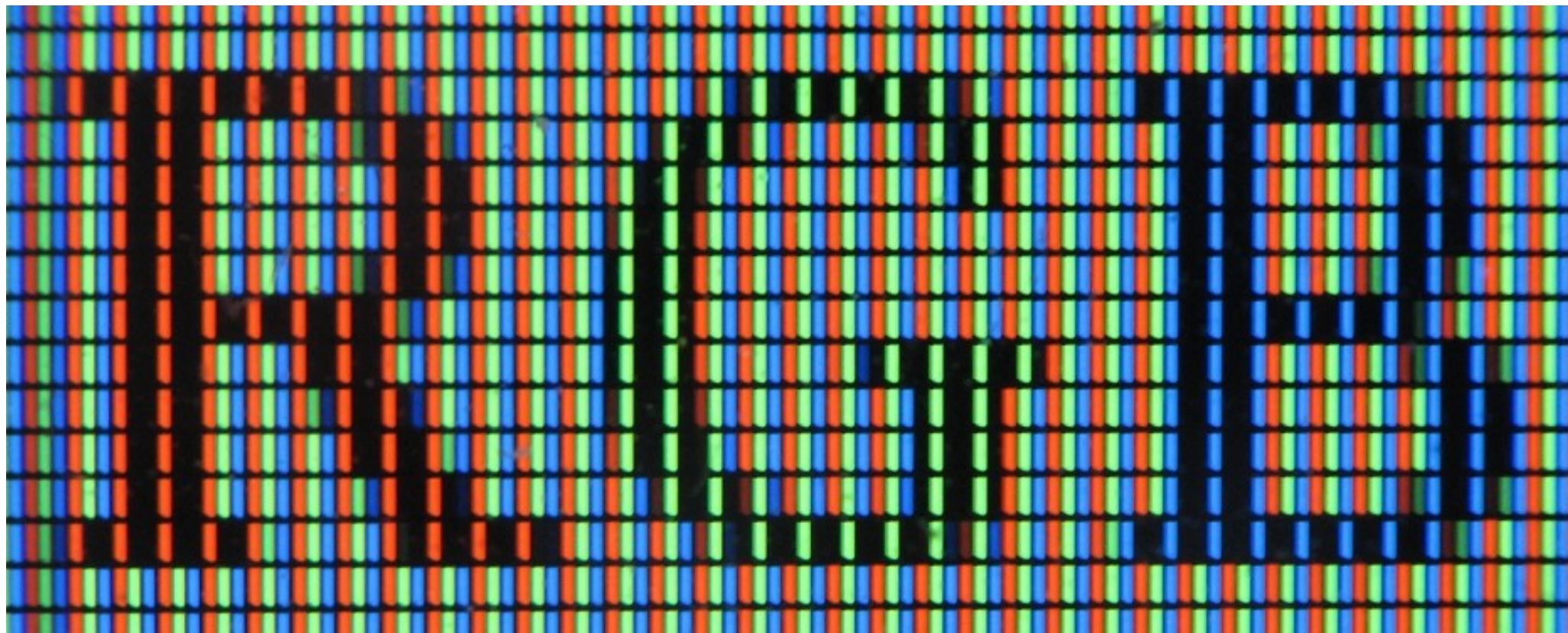
- CRT monitori posjeduju tri elektronska topa koji pogađaju fosforne tačke na zaslonu monitora koje nakon toga emituju svjetlost. Postoje tri tipa tačaka koje prouzrokuju boje iz različitog spektra (rgb). Slično je kod LCD i plazma panela, s razlikom u izvorima RGB svjetlosti.



- Windows API koristi tip COLORREF (**typedef** za 32 bitni broj) za zapis boje



- Makro RGB se može koristiti za kreiranje željene boje sa udjelom osnovnih boja u intervalu 0-255 npr:
 - COLORREF zuta = RGB(255,255,0);



RGB

Tačke, olovke, linije...

`COLORREF SetPixel(HDC hdc, int X, int Y, COLORREF crColor);`

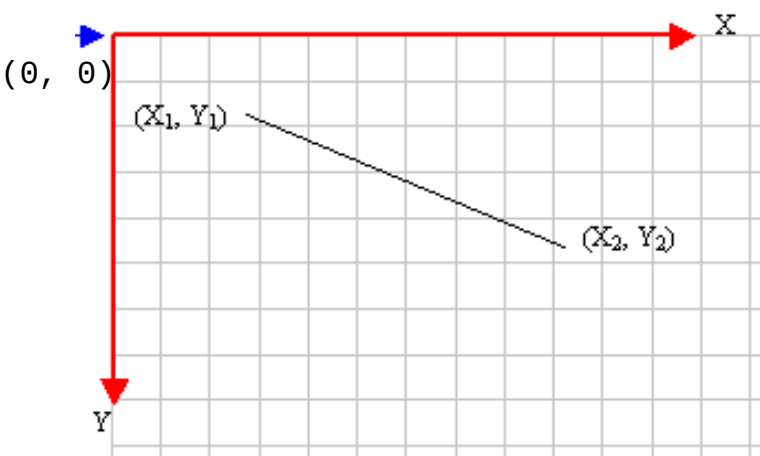
- Boji tačku na koordinati (X,Y) bojom crColor.

`BOOL MoveToEx(HDC hdc, int X1, int Y1, LPPPOINT lpPoint);`

- Postavlja poziciju olovke hdc-a na koordinate (X1,Y1) i upisuje staru poziciju olovke hdc-a u POINT strukturu na adresi lpPoint. Ako nam stara pozicija nije bitna, zadnji parametar postavimo na NULL.
- Povratna vrijednost: nenulta – uspješno, nula – neuspješno.

`BOOL LineTo(HDC hdc, int X2, int Y2);`

- Crta liniju **od** trenutne pozicije hdc-a **do** tačke (X2,Y2), ujedno mijenja trenutnu poziciju olovke u hdc-u na koordinate (X2,Y2).



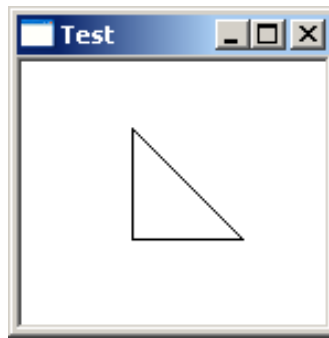
`BOOL Polyline(HDC hdc, const POINT *lpPt, int n);`

- Crta seriju spojenih linija definisanih tačkama koje se nalaze u nizu lpPt. U nizu se nalazi n objekata tipa POINT (sa poljima x i y). Ne koristi trenutnu poziciju olovke.
- Pri crtanju se koristi trenutna boja olovke.

Primjer

```
LRESULT CALLBACK WindProcedure(HWND hWnd, UINT Msg,
    WPARAM wParam, LPARAM lParam)
{
    HDC          hDC;
    PAINTSTRUCT ps;
    switch(Msg)
    {
        case WM_PAINT:
            hDC = BeginPaint(hWnd, &ps);
            MoveToEx(hDC, 50, 80, NULL);
            LineTo(hDC, 100, 80);
            LineTo(hDC, 50, 30);
            LineTo(hDC, 50, 80);
            EndPaint(hWnd, &ps);
            break;
        case WM_DESTROY:
            PostQuitMessage(WM_QUIT);
            break;
    }
    return DefWindowProc(hWnd, Msg, wParam, lParam);
}
```

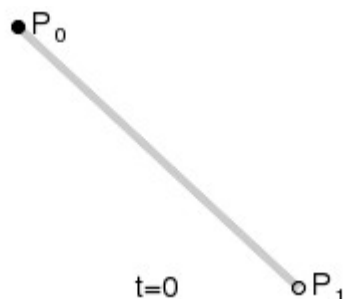
```
LRESULT CALLBACK WindProcedure(HWND hWnd, UINT Msg,
    WPARAM wParam, LPARAM lParam)
{
    HDC          hDC;
    PAINTSTRUCT ps;
    POINT lppt[] = { {50,80}, {100,80}, {50,30}, {50,80} };
    switch(Msg)
    {
        case WM_PAINT:
            hDC = BeginPaint(hWnd, &ps);
            Polyline(hDC, lppt, 4);
            EndPaint(hWnd, &ps);
            break;
        case WM_DESTROY:
            PostQuitMessage(WM_QUIT);
            break;
    }
    return DefWindowProc(hWnd, Msg, wParam, lParam);
}
```



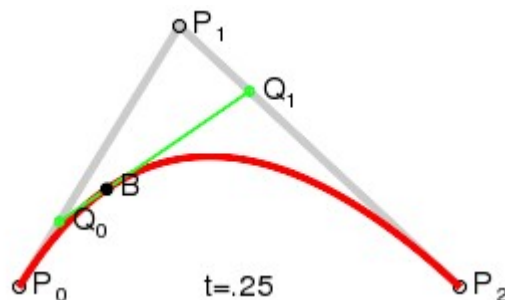
Bezierove krive

- Pierre Etienne Bezier (1.9.1910. – 25.11.1999.)
- Bezierove krive se koriste u vektorskoj grafici za modelovanje glatkih krivih koje se onda mogu skalirati proizvoljno.
 - Renault

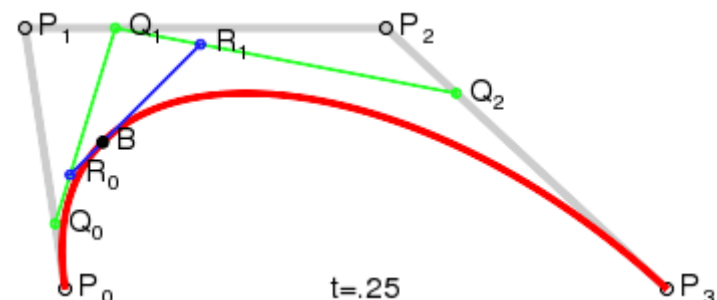
Linearna Bezierova kriva
(linearna interpolacija)



Kvadratna Bezierova kriva



Kubna Bezierova kriva



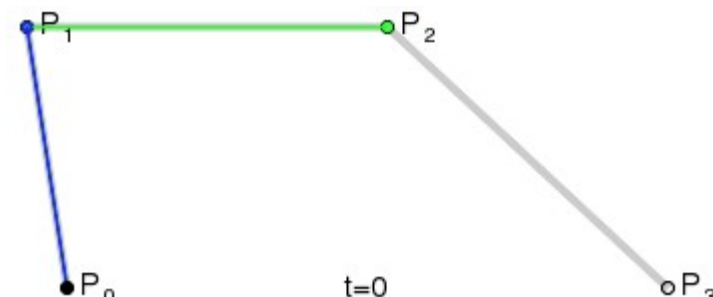
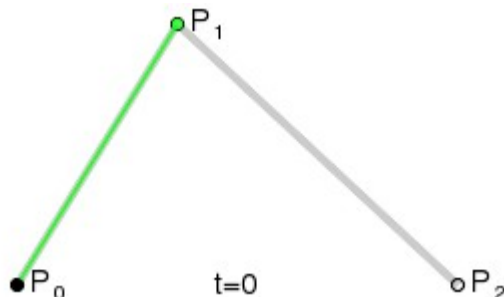
$$P_0(x_0, y_0), P_1(x_1, y_1)$$

$$\Delta x = x_1 - x_0$$

$$\Delta y = y_1 - y_0$$

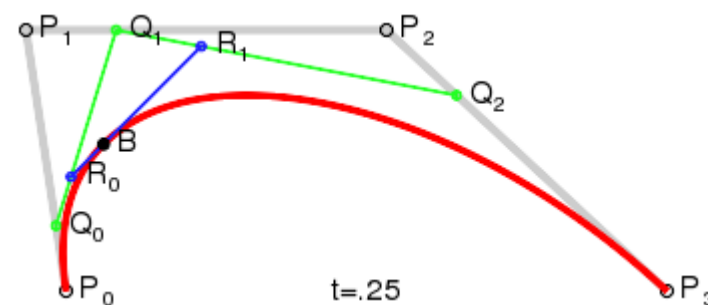
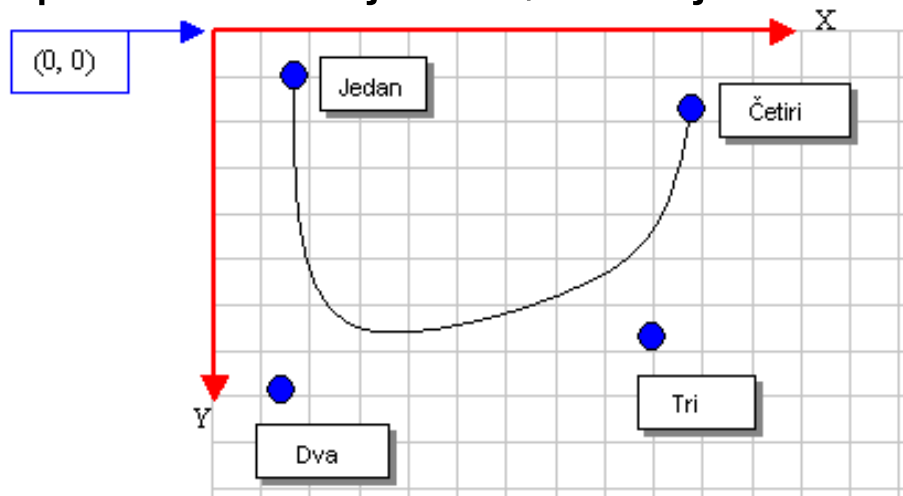
$$x_t = x_0 + t \cdot \Delta x$$

$$y_t = y_0 + t \cdot \Delta y$$



Bezierove krive

- Za crtanje kubne Bezierove krive potrebne su četiri tačke: dvije za početak i kraj krive, te dvije kontrolne tačke.



BOOL PolyBezier(HDC hdc, CONST POINT *lppt, DWORD n);

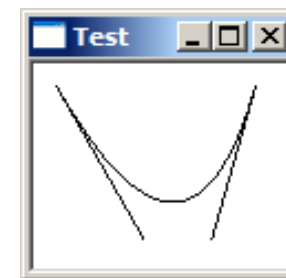
- `lppt` je niz objekata tipa `POINT` koji predstavljaju tačke Bezierovih krivih na način da je prva tačka početak prve krive, druga i treća su kontrolne tačke a četvrta je krajnja tačka prve krive; svaka naredna kriva je određena sa dodatne tri tačke. Kraj prve krive je početak za narednu itd. Trenutna pozicija olovke se ne mijenja.
- `n` je broj tačaka u nizu. ($3 \times \text{broj krivih} + 1$)

Primjer

```

LRESULT CALLBACK WndProc(HWND hWnd, UINT Msg, WPARAM wParam, LPARAM lParam)
{
    HDC hDC;
    PAINTSTRUCT ps;
    POINT Pt[4] = { { 10, 10 }, { 50, 80 }, { 80, 80 }, { 100, 10 } };
    switch(Msg)
    {
        case WM_PAINT:
            hDC = BeginPaint(hWnd, &ps);
            PolyBezier(hDC, Pt, 4);
            MoveToEx(hDC, Pt[3].x, Pt[3].y, NULL);
            LineTo(hDC, Pt[2].x, Pt[2].y);
            MoveToEx(hDC, Pt[0].x, Pt[0].y, NULL);
            LineTo(hDC, Pt[1].x, Pt[1].y);
            EndPaint(hWnd, &ps);
            break;
        case WM_DESTROY:
            PostQuitMessage(WM_QUIT);
            break;
    }
    return DefWindowProc(hWnd, Msg, wParam, lParam);
}

```



BOOL PolyBezierTo(HDC hdc, CONST POINT *lppt, DWORD n);

Radi sve kao i PolyBezier ali dodatno postavlja poziciju olovke na zadnju tačku u nizu.

Olovka (PEN)

DC koristi objekte tipa olovke za crtanje linija i krivih. Za crtanje proizvoljnom olovkom umjesto olovkom koju trenutno koristi DC potrebno je izvršiti korake:

1. Kreirati tip olovke po potrebi:
 - pomoću jedne od funkcija: `CreatePen` ili `CreatePenIndirect`.
2. Postaviti (odabrati, “selektovati”) kreiranu olovku u DC pomoću funkcije `SelectObject`, a sačuvati staru olovku
 - funkcijom `SelectObject` novi objekat se postavlja umjesto postojećeg objekta istog tipa. Funkcija vraća handle na postojeći objekat.
3. Izvršiti poziv za crtanje u datom DC.
4. Selektovati olovku van DC-a i vratiti staru olovku u DC.
5. Izbrisati kreiranu olovku.

Funkcije za kreiranje olovke

HPEN CreatePen(int stil, int debljina, COLORREF boja);

- Vraća handle na olovku željenog stila, debljine i boje. Validni stilovi su:

PS_SOLID	_____
PS_DASH	- - - - -
PS_DOT	
PS_DASHDOT	- . - . - .
PS_DASHDOTDOT	- . - . - . .
PS_NULL	
PS_INSIDEFRAME	_____

- Alternativno moguće je prvo definisati strukturu tipa LOGPEN te popuniti njena polja željenim parametrima, a zatim proslijediti njenu adresu funkciji CreatePenIndirect:

HPEN CreatePenIndirect(CONST LOGPEN *lplogpn);

```
typedef struct tagLOGPEN {
    UINT lopnStyle;
    POINT lopnWidth;
    COLORREF lopnColor;
} LOGPEN, *PLOGPEN;
```

Selektovanje i brisanje olovaka

HGDI OBJ SelectObject(HDC hdc, HGDI OBJ hObject);

- Funkcija selektuje objekat hObject u hdc a vraća handle na objekat kojeg je hObject zamijenio u DC-u.

BOOL DeleteObject(HGDI OBJ hObject);

- Svaki kreirani objekat (npr. olovka) potrebno je izbrisati nakon upotrebe.
 - Za brisanje je dovoljno proslijediti handle na objekat koji se briše.
 - Nije ispravno brisati objekat koji je trenutno selektovan u DC-u.
 - Ako se objekat ne izbriše sistem ga čuva do kraja života aplikacije.

Stock objekti

- Windows OS predefinisano drži određene tipove objekata na “zalihamu” (in stock), pa tako i određene tipove olovaka. Funkcija za dobijanje Stock objekta je:

```
HGDIOBJ GetStockObject (int fnObject);
```

- fnObject je tip objekta, npr. za olovke je WHITE_PEN, BLACK_PEN i NULL_PEN.

```
HPEN hPen = (HPEN) GetStockObject(WHITE_PEN);
```

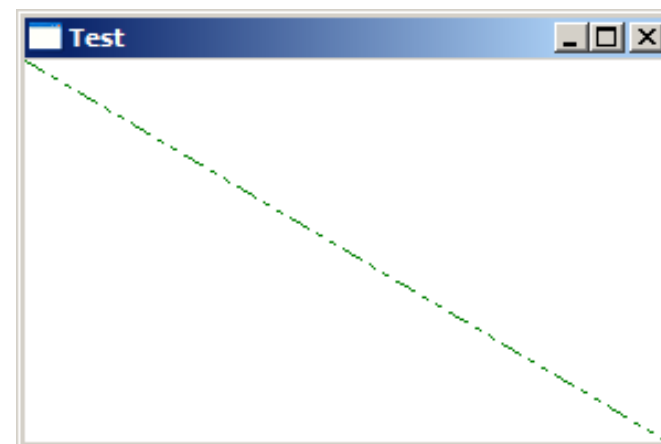
- Stock objekte nije potrebno brisati nakon upotrebe.

Primjer

```

LRESULT CALLBACK WndProc (HWND hwnd, UINT message, WPARAM wParam, LPARAM
lParam)
{
    static int  cxClient, cyClient;
    HDC hdc;
    PAINTSTRUCT ps;
    HPEN stara_olovka, nova_olovka;
    switch (message)
    {
        case WM_SIZE:
            cxClient = LOWORD(lParam);
            cyClient = HIWORD(lParam);
            return 0;
        case WM_DESTROY:
            PostQuitMessage (0) ;
            return 0 ;
        case WM_PAINT:
            hdc = BeginPaint(hwnd, &ps);
            nova_olovka = CreatePen(PS_DASHDOTDOT, 1, RGB(0, 128, 0) );
            stara_olovka = SelectObject(hdc, nova_olovka);
            LineTo(hdc, cxClient, cyClient);
            SelectObject(hdc, stara_olovka);
            DeleteObject(nova_olovka);
            EndPaint(hwnd, &ps);
            return 0;
    }
    return DefWindowProc (hwnd, message, wParam, lParam) ;
}

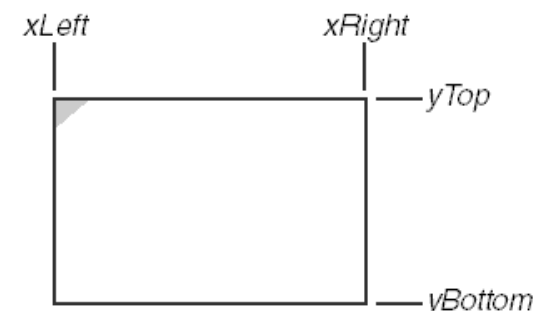
```



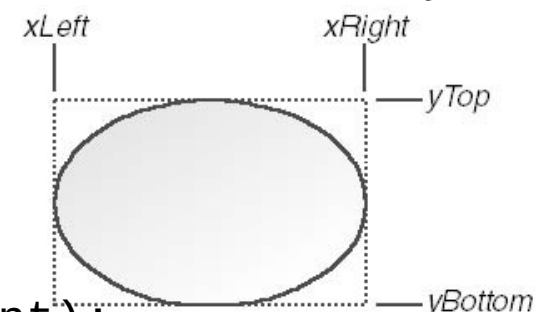
Područja

- Windows-i posjeduju funkcije za crtanje zatvorenih područja. Obod područja crta se trenutnom **olovkom** selektovanom u DC a unutrašnjost se boji trenutnom **četkom** selektovanom u DC. Ove funkcije ne zavise od trenutne pozicije olovke:

`Rectangle (hdc, xLeft, yTop, xRight, yBottom);`



`Ellipse (hdc, xLeft, yTop, xRight, yBottom);`



`BOOL Polygon(HDC hdc, CONST POINT *lpPoints,int nCount);`

- Crta poligon od nCount tačaka definisanim u nizu lpPoints.
- Za crtanje više poligona koristiti funkciju:

`BOOL PolyPolygon(HDC hdc, CONST POINT *lpPoints, CONST INT *lpPolyCounts, int nCount);`

Četke

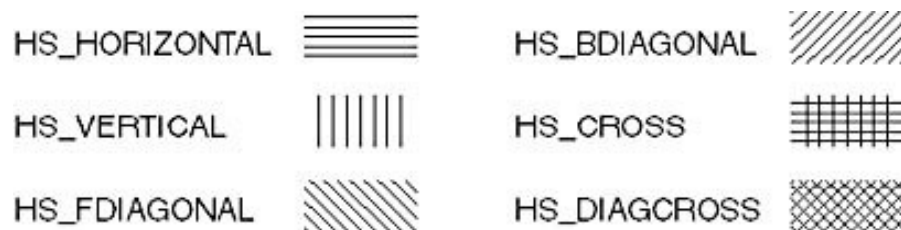
- Četke su objekti koje DC koristi za bojenje zatvorenih područja. Kao i olovke potrebno ih je prije korištenja kreirati i selektovati u DC, a nakon korištenja uništiti. Za kreiranje se mogu koristiti naredne funkcije:

```
HBRUSH CreateSolidBrush(COLORREF crColor);
```

- Vraća handle na četku crColor boje.

```
HBRUSH CreateHatchBrush( int fnStyle, COLORREF clrref);
```

- Vraća handle na četku sa uzorkom fnStyle i bojom clrref. Validni uzorci su:



- Moguće je dobiti handle na stock četku kao npr:

```
HBRUSH hBrush = (HBRUSH) GetStockObject(GRAY_BRUSH);
```

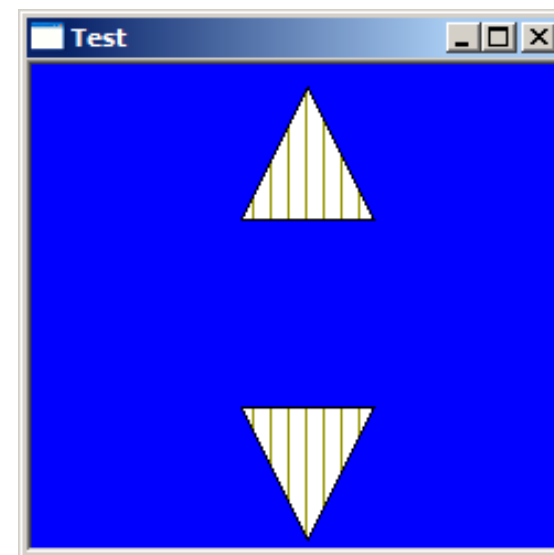
Primjer

```

LRESULT CALLBACK WndProc(HWND hWnd, UINT Msg, WPARAM wParam, LPARAM lParam)
{
    HDC hDC;
    HBRUSH hbKlijent, hbTrougao;
    PAINTSTRUCT Ps;
    RECT rect;
    POINT Pt[] =
    {
        {125, 10},
        {95, 70},
        {155, 70},

        {95, 155},
        {125, 215},
        {155, 155}
    };
    int lpPts[] = { 3, 3 };
    switch(Msg)
    {
        case WM_PAINT:
            hDC = BeginPaint(hWnd, &Ps);
            hbTrougao = CreateHatchBrush(HS_VERTICAL, RGB(150, 150, 0));
            hbKlijent = CreateSolidBrush(RGB(0, 0, 255));
            SelectObject(hDC, hbTrougao);
            GetClientRect(hWnd, &rect);
            FillRect(hDC, &rect, hbKlijent);
            PolyPolygon(hDC, Pt, lpPts, 2);
            EndPaint(hWnd, &Ps);
            DeleteObject(hbKlijent);
            DeleteObject(hbTrougao);
            break;
        case WM_DESTROY:
            PostQuitMessage(WM_QUIT);
            break;
    }
    return DefWindowProc(hWnd, Msg, wParam, lParam);
}

```



GetClientRect u strukturu rect postavlja vrijednost koja odgovara pravougaoniku klijent područja.

FillRect puni područje u hDC, određeno pravougaonikom rect, četkom hbKlijent a ne trenutnom četkom u hDC-u.